Steganography

The "What" (Introduction)

❖ What is it?

Steganography is the art of hiding a secret message within an ordinary, non-suspicious carrier, so that the very existence of the message remains concealed.

➢ The “Ancient World”

As early as 440 BC, according to Greek historian Herodotus, ruler Histiaeus would shave the head of his trusted slave, then tattoo the secret message on the slave's scalp and then would wait for the hair to grow back. Once the hair is all grown, the slave would be sent to the destination to deliver the message without being suspected of carrying a message.

The Spartan king Demaratus sent a warning about a planned Persian invasion by scraping the wax off a wooden writing tablet, carving his message into the wood, and then covering it with a fresh layer of wax. The blank-looking tablet was then transported without raising suspicion.

➢ Middle Ages to 19th Century, The Development Phase

By 3rd Century BC, the Greek Scholar Philo of Byzantium was among the first to write about an invisible ink to hide messages, using a recipe comprising of oak gall nuts and copper sulfate solution.

The term steganography itself was first coined in year 1499 by the German abbot Johannes Trithemius in his book Steganographia. This

work was disguised as a work of magic while it was a treatise on Cryptography and Steganography. Due to its association with magic, The Catholic Church placed this book on the list of forbidden books in the year 1609.

In 1605, the English philosopher Francis Bacon devised a method to hide messages by using two slightly different typefaces within a text. This is an early example of encoding information in the visual presentation of a message

- The 20th Century: Espionage and War

Microdots were developed in the late 19th century but heavily used in World War I and II. Microdots involved shrinking a photograph of a message down to the size of a period. These dots were then hidden on postcards, in letters, or in other documents. The Germans used them extensively in WWII, and the Allies discovered the practice in 1941.

During the wars, both the French Resistance and Axis spies used invisible inks, and methods like using Morse code in knitting patterns were also employed

During the Vietnam War, U.S. Navy pilot Jeremiah Denton, while a prisoner of war and being filmed for propaganda, blinked his eyes in Morse code to spell out "T-O-R-T-U-R-E," a desperate act to send a hidden message to his countrymen.

In another example, there were rumors that during the 1980s Margareth Thatcher, then Prime Minister in UK, became so irritated about press leaks of cabinet documents, that she had the word processors programmed to encode the identity of the writer in the word spacing, thus being able to trace the disloyal ministers.

- The Digital Era

The shift from physical to digital media transformed the field. While the first digital steganographic systems emerged around 1985, the widespread adoption of the internet in the 1990s caused an explosion in its use and development

Today, messages are hidden in the least significant bits of images, audio files, video, and network packets. The subtle changes are imperceptible to human senses, allowing for covert communication on a massive scale

Digital steganography is now used for a wide range of purposes, from digital watermarking to protect copyright and track ownership, to embedding patient information in medical images. However, its secrecy also makes it a tool for illicit activities, as suspected use by terrorists and criminals has been a major concern, especially after 9/11.

The advancement of digital steganography has also spurred the creation of steganalysis—the practice of detecting and deciphering hidden messages. The two fields drive each other forward, as new hiding methods are met with new detection techniques.

➢ This Project

While historical steganography techniques relied on physical mediums like wax tablets, invisible inks, and microdots, modern digital steganography operates at the bit level. This project explores the practical implementation of a multi-carrier steganography engine, capable of hiding encrypted messages inside text (using Zero-Width Characters), images (using LSB modification of RGB pixels), and audio (using LSB modification of audio samples). By combining encryption (Fernet/AES-128) with steganographic embedding, the system provides both secrecy (the message is hidden) and security (if

discovered, the message remains unreadable without the key). The project aims to demonstrate not only how steganography works, but also its limitations, detectability, and real-world feasibility in an era of pervasive surveillance.

The "Why" (Purpose)

❖ Why does it matter?

Throughout history, there has been a persistent need to communicate secretly, now whether that be to transmit a strategic military message across enemy territory, or to securely send a password to a web server to verify one's identity and gain access to an account. A message is considered "secret" precisely because its disclosure to an adversary could cause harm, compromise operations, or negatively affect one's dealings. This necessity has driven the evolution of secret communication techniques for millennia.

In the modern digital era, billions of people communicate online every day. This communication encompasses everything from government policy announcements to cover intelligence operations, from password sharing with web servers to private personal conversations. However, this digital landscape introduces a critical threat: the eavesdropper.

An eavesdropper, often modelled in security literature as an adversary sitting between two communicating parties, can intercept communications, access sensitive information, and use that knowledge to adversely affect one or both parties for their own self-interest. This is the classic Man-in-the-Middle (MitM) attack scenario. The threat is not theoretical; it is a daily reality on the internet, from unsecured Wi-Fi networks to sophisticated state-sponsored surveillance.

Given this pervasive threat, it becomes crucial to develop new methods for secret and secure online communication. Encryption provides one layer of

protection by scrambling the content of a message, but it does not hide the fact that a message is being sent. This is where steganography enters the picture.

Steganography addresses a fundamental gap in traditional cryptography: plausible deniability. If an adversary cannot even prove that a secret message exists, they cannot compel you to decrypt it. Hiding the message within an innocent looking carrier, like an image, an audio file, or a text document, steganography allows parties to communicate covertly, evading detection by network monitors, corporate firewalls, and surveillance systems.

In this project, we combine both approaches: encrypting the message first (to protect its content) and then hiding it using steganography (to protect its existence). This provides a layered defense against adversaries, ensuring that even if the hidden data is discovered, it remains unreadable without the encryption key.

❖ Cryptography vs Steganography

Cryptography and steganography are two distinct approaches to securing information, each with its own strengths, weaknesses, and threat models.

➢ Cryptography

Cryptography is the practice of transforming a plaintext message into an unreadable format called ciphertext, using a mathematical algorithm (cipher) and a secret key. The ciphertext is transmitted over an insecure channel. The intended recipient, possessing the correct key, applies the reverse algorithm to recover the original message.

For example, a simple Caesar cipher shifts each letter by a fixed number: "HELLO" becomes "KHOOR" with a shift of 3. Modern encryption algorithms like AES-256 are vastly more complex, but the

fundamental principle remains the same: the message is scrambled so that it is unintelligible to anyone who intercepts it without the key.

The primary challenge of cryptography is key distribution, securely sharing the key between the sender and recipient without it being intercepted by an adversary. If the key is compromised, the encryption is useless. Additionally, encryption does not hide the fact that communication is occurring; an eavesdropper can see that a secret message is being sent and may attempt to break the cipher or compel the sender to reveal the key.

- Steganography

Steganography, in contrast, operates on a fundamentally different ideology. Rather than scrambling the message, steganography conceals its existence entirely by embedding it within an innocent looking carrier file, like an image, audio clip, or text document. The carrier appears perfectly normal to anyone who observes it; the secret remains hidden in plain sight.

The primary challenge of steganography is capacity and detectability. The carrier must be large enough to hold the secret without introducing perceptible changes that might raise suspicion. Additionally, sophisticated adversaries may employ steganalysis, a statistical technique designed to detect the presence of hidden data.

Feature	**Cryptography**	**Steganography**
Goal	Hide the content of the message.	Hide the existence of the message.
Output	Scrambled, unreadable ciphertext.	A perfectly normal looking carrier file.
Attack Model	The attacker knows that a secret is being	The attacker does not know that a

	sent and tries to decipher it.	secret is being sent.
Primary Challenge	Key distribution.	Capacity and detectability.
Weakness	Does not hide the fact that communication is occurring.	Vulnerable to statistical steganalysis.
Analogy	Putting a letter in a locked safe.	Writing a secret message with an invisible ink on a postcard.

❖ Why LSB?

All forms of digital media seen over the internet, on your phone, or anywhere are made up of bits. Now, what is a bit? For simplicity, a bit is just a switch which goes "on" and "off". Now a curious mind might ask, how can an image, an audio, or a text like "hello" be made up of switches that simply go "on" and "off"? That's another great question. Let's handle the three beasts of text, image, and audio one at a time.

Computer scientists a couple of decades ago manufactured a piece of technology called computers, which had the capacity to store lots of these switches. The scientists then decided to use a sequence of these switches to represent each character in the English alphabet. For example, the alphabet "A" is represented in a computer as a sequence of switches as follows: "01000001", where 0 means "off", and 1 means "on". These unique sequences for all the English alphabets were assigned by computer scientists, which made text storable in the computer. Therefore, a simple "Hi" on the computer looks like the following: "10010000 (H) 01101001 (i)". Similarly, scientists also assigned unique bit sequences for grammatical characters, where the space character is represented as "00100000" in computers.

In the same way, bits are used to store audio. When we record an audio sample, what we are really recording is the frequency of air pressure sensed by the microphone. Since frequencies are represented as numbers, computers store those numbers as a sequence of bits as well, where each sequence corresponds to a specific number. The sequence of bits storing a number is said to represent a number with base 2. Humans on a daily basis use base 10 to store numbers and can easily convert numbers from base 10 to base 2. In this way, computers store audio.

Now comes the last beast of the trio: the image. Each image we encounter, whether digital or painted, is built out of colors. In the same way a painter uses colors to draw a picture; computers store pictures as a series of colors. It may sound absurd, but if a picture is divided into a million small pieces, each piece on its own would simply look like a shade of a color, and it's when we put these pieces together like in a puzzle game that they look like the picture. A computer does the same thing: it breaks an image into millions of pieces, where each piece is called a pixel. It uses a color gradient which assigns each shade of a color with a value. Since all colors are made up of the elementary colors red, green, and blue, computers store the specific values assigned to the shades of these three colors used to make the specific color of the pixel. And that's how a computer stores an image.

Now, in all three scenarios we defined above, each had a sequence of switches, or bits, involved. The bit at the right-most position of the sequence is called the least significant bit or LSB. It is called that because, like in our normal world counting, the digit in the right-most position is the one that holds the lowest value. For example, in the number 10096, the '6' is the digit that represents the smallest quantity (6 units). The '9' represents 90, and gradually the value gets bigger as you go to the left. The same is the case with bit sequences; they are just numbers. Like how we use 0-9 digits to represent a number, bits represent any number with just digits 0 and 1.

So, why do we specifically use LSB for steganography?

The answer lies in imperceptibility.

Think about changing numbers in the real world. If you take the number 10096 and change the right-most digit from a '6' to a '7', the number becomes 10097, a tiny, almost insignificant difference. But if you change the left-most '1' to a '2', the number becomes 20096, a massive, obvious change!

The same logic applies to digital media. The LSB holds the smallest value in a pixel's color or an audio sample's frequency. By changing just this single bit (from 0 to 1 or 1 to 0), we alter the color or sound by an almost imperceptible amount, a shade so slight that the human eye or ear cannot detect it.

This makes the LSB the perfect hiding spot. A stenographer can overwrite the LSBs of millions of pixels in an image or samples in an audio file with the bits of their secret message. To any viewer or listener, the image or audio appears completely normal. However, anyone who knows the secret can extract those tiny, altered LSBs, piece them back together, and reveal the hidden data. It's the digital equivalent of hiding a letter in the faintest, most invisible ink imaginable!

The "How" (Design and Architecture)

- High-level Game Plan

Steganography requires two things: a secret message to hide, and a carrier medium to hide it in. Depending on the medium-text, image, or audio-each implementation differs, but the core pipeline remains the same:

1. Input: The user provides a secret message and a carrier file (text, image, or audio).
2. Encrypt: The secret is encrypted using Fernet (AES-128) with a randomly generated key.

3. Binary Conversion: The encrypted bytes are converted into a binary string of 1s and 0s.
4. Header Addition: A 32-bit header containing the size of the secret (in bits) is prepending on the binary data.
5. Embedding: The binary data (header + secret) is embedded into the LSBs of the carrier file.
6. Extraction: The process is reversed; LSBs are extracted; the header is read, and the secret is decrypted.

- Hiding in Text

  Concept:

  Zero-Width Characters are invisible Unicode characters that are rendered as empty space. Two such characters are used in this project:

  \u200B (Zero-Width Space) = Bit 1
  \u200C (Zero-Width Non-Joiner) = Bit 0

  For example, the string "Hi" with an invisible character appears as:
  01001000 (H) 01101001 (i) 11100010 10000000 10001011 (Zero-Width Space)

  To the human eye, it looks exactly like "Hi". By inserting these invisible characters between the visible letters of a cover text, we can embed our encrypted secret without altering the visible appearance of the text.

  Code Implementation:

  The encoder loops through the cover text (mask_text) and appends an invisible character after each visible character based on the next bit of the encrypted secret:

```
for char in mask_text:
            final_message.append(char)

            if (bit_index < len(binary_secret)):
                if (binary_secret[bit_index] == '1'):
                    final_message.append(CHAR_ONE)
                elif (binary_secret[bit_index] == '0'):
                    final_message.append(CHAR_ZERO)
                bit_index += 1
```

The decoder reverses this process: it scans the text, extracts the invisible characters, reconstructs the binary string, and decrypts the result. When implementing this technique, a header could be avoided completely, since one could mark the last zero-width character as the end of the secret message, making that saved space of header being used to transfer more data.

Evidence:

- Hiding text

```
PS D:\comp6441\Awesome Project> python3 .\src.py
 _____ __                                                    __
/ ___// /____ ____ _____ _____ ____ ____ _________ _____ / /_  __  __
\__ \/ __/ _ \/ __ `/ __ `/ __ \/ __ \/ __ `/ ___/ __ `/ __ \/ __ \/ / / /
___/ / /_/  __/ /_/ / /_/ / / / / /_/ / /_/ / /  / /_/ / /_/ / / / / /_/ /
/____/\__/\___/\__, /\__,_/_/ /_/\____/\__, /_/   \__,_/ .___/_/ /_/\__, /
              /____/                  /____/         /_/           /____/

Press 1 to hide secrets in text.
Press 2 to hide secrets in an image.
Press 3 to hide secret in an audio file.
Press 4 to decode from a text.
Press 5 to find a secret from a image.
Press 6 to find the secret from an audio file.
To exit press 7.

Please enter your choice: 1


Enter your secret message: I hate Kanye West
Enter the mask text: I like Kanye West


USING ZERO WIDTH STEGNOGRAPHY


This is the key for text stegnography: QTVA19LPPGCQoNiHKffvDhnLkhDdc-5dWu4TeIFASEY=

Original text: I hate Kanye West
Encrypted bytes (gibberish): gAAAAABqYSg4WlsHy4MxE8kYRZHHqQbCpVlHctlxJkFTuSJUqKqyuE9nHB_ac8qEfdcmF91TlmbKQDVXOZANsT_y2-4uqMJ9dYNCfvufewTdNLAaPgb8i5I=
Final message:  I like Kanye West
Total characters in final text (visible + invisible): 977
```

- Finding secret

```
 _____ __                                                    __
/ ___// /____ ____ _____ _____ ____ ____ _________ _____ / /_  __  __
\__ \/ __/ _ \/ __ `/ __ `/ __ \/ __ \/ __ `/ ___/ __ `/ __ \/ __ \/ / / /
___/ / /_/  __/ /_/ / /_/ / / / / /_/ / /_/ / /  / /_/ / /_/ / / / / /_/ /
/____/\__/\___/\__, /\__,_/_/ /_/\____/\__, /_/   \__,_/ .___/_/ /_/\__, /
              /____/                  /____/         /_/           /____/

Press 1 to hide secrets in text.
Press 2 to hide secrets in an image.
Press 3 to hide secret in an audio file.
Press 4 to decode from a text.
Press 5 to find a secret from a image.
Press 6 to find the secret from an audio file.
To exit press 7.

Please enter your choice: 4


Enter the text from which you wanna extract the secret from: I like Kanye West
Enter the key for decryption: QTVA19LPPGCQoNiHKffvDhnLkhDdc-5dWu4TeIFASEY=


USING ZERO WIDTH STEGNOGRAPHY DECODER


Mask text:  I like Kanye West
Real (Secret Message):  I hate Kanye West
```

- Hiding in Image

  Concept:

Images are composed of pixels, each with three color channels: Red, Green, and Blue. Each channel is an 8-bit value (0-255). By modifying the Least Significant Bit (LSB) of each channel, we can hide 3 bits of data per pixel without any perceptible change to the image.

For example:

200 in binary is 11001000 (LSB = 0).
201 in binary 11001001 (LSB = 1).

Changing 200 to 201 is invisible to the human eye but encodes one bit of information.

Code Implementation:

The image is loaded using PIL and converted to RGB format. The pixel data is flattened into a 1D NumPy array, and each byte's LSB is modified:

```
# Open image and convert it into an array of bytes
        img = Image.open(input_img_path)
        pixels = np.array(img)
        original_shape = pixels.shape
        pixels = pixels.flatten()

        # Get the size of input in bits
        if (len(secret_bytes) > (len(pixels) - HEADER_SIZE)):
            print("SIZE ERROR: Image too small to hide the text!")
            return

        # Convert secret text size into raw bytes string
        size_bytes = len(secret_bytes).to_bytes(4, byteorder="big")
        size_string = ''.join(format(byte, "08b") for byte in size_bytes)

        data_load = size_string + secret_bytes

        byte_index = 0
        for char in data_load:
```

```
            if (char == "1"):
                pixels[byte_index] = pixels[byte_index] | 1
            elif (char == "0"):
                pixels[byte_index] = pixels[byte_index] & 0xFE
            byte_index += 1

        pixels = pixels.reshape(original_shape)
        output_image = Image.fromarray(pixels)

        if not output_img_path.lower().endswith((".png")):
            base, ext = os.path.splitext(output_img_path)
            output_img_path = base + ".png"

        output_image.save(output_img_path)
        print(f"[+] Stego image saved: {output_img_path}")
```

Evidence:

- Hiding secret

```
 _____ __                                                          __
/ ___// /____  ____ _____ _____  ____  ____ _________ _____  / /_  __  __
\__ \/ __/ _ \/ __ `/ __ `/ __ \/ __ \/ __ `/ ___/ __ `/ __ \/ __ \/ / / /
___/ / /_/  __/ /_/ / /_/ / / / / /_/ / /_/ / /  / /_/ / /_/ / / / / /_/ /
/____/\__/\___/\__, /\__,_/_/ /_/\____/\__, /_/   \__,_/ .___/_/ /_/\__, /
              /____/                  /____/         /_/         /____/

Press 1 to hide secrets in text.
Press 2 to hide secrets in an image.
Press 3 to hide secret in an audio file.
Press 4 to decode from a text.
Press 5 to find a secret from a image.
Press 6 to find the secret from an audio file.
To exit press 7.

Please enter your choice: 2


Enter your secret message: I think I hate Kanye, like a lot
Enter the path for image to hide text in: d:\comp6441\Awesome Project\testImage.jpg
Enter the path for the out image: d:\comp6441\Awesome Project\resultImage.jpg


USING IMAGE STEGNOGRAPHY


This is the key for image stegnography: 4e0PBsO-SRXMrk4uJHIHiMCwarrJmVUp5hDu2bsqbCA=

[+] Stego image saved: d:\comp6441\Awesome Project\resultImage.png
```

- Finding secret

```
 _____ __                                                          __
/ ___// /____  ____ _____ _____  ____  ____ _________ _____  / /_  __  __
\__ \/ __/ _ \/ __ `/ __ `/ __ \/ __ \/ __ `/ ___/ __ `/ __ \/ __ \/ / / /
___/ / /_/  __/ /_/ / /_/ / / / / /_/ / /_/ / /  / /_/ / /_/ / / / / /_/ /
/____/\__/\___/\__, /\__,_/_/ /_/\____/\__, /_/   \__,_/ .___/_/ /_/\__, /
              /____/                  /____/         /_/         /____/

Press 1 to hide secrets in text.
Press 2 to hide secrets in an image.
Press 3 to hide secret in an audio file.
Press 4 to decode from a text.
Press 5 to find a secret from a image.
Press 6 to find the secret from an audio file.
To exit press 7.

Please enter your choice: 5


Enter the path for image to extract text from: d:\comp6441\Awesome Project\resultImage.png
Enter the key for decryption: 4e0PBsO-SRXMrk4uJHIHiMCwarrJmVUp5hDu2bsqbCA=


USING IMAGE STEGNOGRAPHY DECODER


The secret message is: I think I hate Kanye, like a lot
```

- Input picture

- Output Picture

- Hiding in Audio

Concept:

WAV audio files consist of a 44-byte header followed by raw audio samples. Each sample is a 16-bit integer representing the amplitude of the sound wave at a specific moment in time. By modifying the LSB of each sample, we can hide 1 bit of data per sample without any perceptible change to the audio.

Code Implementation:

The WAV file is loaded using Python's wave module. The raw frames are converted to a NumPy array of 16-bit integers, and the LSB of each sample is modified:

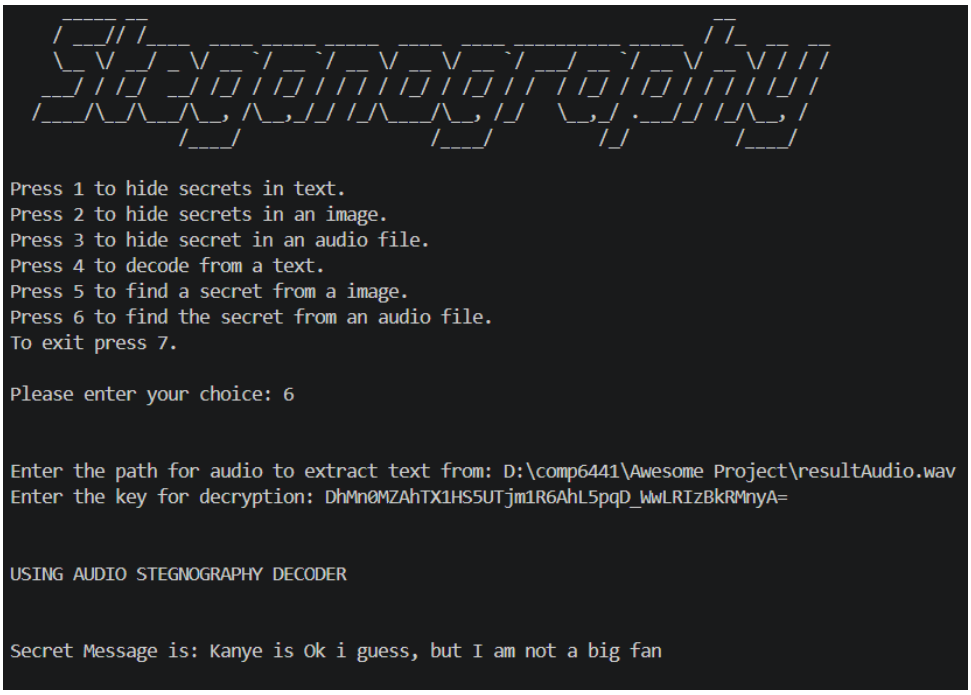
```
with wave.open(input_audio_path, 'rb') as wav:
            # 1. Read the header parameters
            params = wav.getparams()

            nchannels, sampwidth, framerate, nframes = params[:4]

            if (len(secret_byte) > nframes - HEADER_SIZE):
                print("SIZE ERROR: Audio is too small to hide the secret in it!")
                return None

            # 2. Read ALL audio data as raw bytes
            frames = wav.readframes(nframes)
            samples = self.read_samples(frames, sampwidth)

            # convert the size of encrypted secret into bytes array
            size_byte = len(secret_byte).to_bytes(4, byteorder="big")
            size_array = bytearray()
            for byte in size_byte:
                for i in range(7, -1, -1):
                    bit = (byte >> i) & 1
                    size_array.append(bit)

            size_array.extend(secret_byte)

            # embed the secret message into the audio file
            byte_index = 0
```

```
            for bit in size_array:
                if (bit):
                    samples[byte_index] = samples[byte_index] | 1
                else:
                    samples[byte_index] = samples[byte_index] & 0xFE
                byte_index += 1

            # convert modified integer array frames for wav file
            modified_frames = self.write_samples(samples, sampwidth)

            # store the new data into a file
            with wave.open(output_audio_path, 'wb') as wav:
                wav.setparams(params)
                wav.writeframes(modified_frames)
```

Evidence

- Hiding Secret

- Finding Secret



- The Header (Size Tracking)

To allow the decoder to know exactly when to stop reading bits, a 32-bit header containing the size of the secret (in bits) is embedded in the first 32 positions of the carrier:

```
# convert the size of encrypted secret into bytes array
            size_byte = len(secret_byte).to_bytes(4, byteorder="big")
            size_array = bytearray()
            for byte in size_byte:
                for i in range(7, -1, -1):
                    bit = (byte >> i) & 1
                    size_array.append(bit)

            size_array.extend(secret_byte)
```

Without this header, the decoder would not know how many bits to extract, making extraction impossible.

❖ Extraction (the find method)

The extraction process is the reverse of embedding:

1. Load the carrier file.
2. Extract the first 32 LSBs to recover the size header.
3. Extract exactly size bits from the remaining LSBs.
4. Convert the bits back to bytes.
5. Decrypt using the key.
6. Return the decrypted secret.

```
counter = 0
        encrypted_size = []
        while (counter < HEADER_SIZE):
            encrypted_size.append(str(pixels[counter] & 1))
            counter += 1

        size = int(''.join(encrypted_size), 2)

        end_count = size + counter
```

```
        encrypted_secret = []
        while (counter <  end_count):
            encrypted_secret.append(str(pixels[counter] & 1))
            counter += 1

        secret_num = int(''.join(encrypted_secret), 2)
        encrypted_secret = secret_num.to_bytes((len(encrypted_secret) + 7) // 8,
byteorder='big')
        secret = cipher.decrypt(encrypted_secret).decode()

        print("The secret message is: " + secret)
        return secret\
```

❖ Alternative Methods (Not Implemented)
While this project focuses on Zero-Width Characters and LSB steganography, other text-based methods exist:

- Synonym Substitution: Replacing words with synonyms (e.g., "big" = 1, "large" = 0).
- Passive/Active Voice: Using grammatical voice to encode bits (e.g., passive voice = 1, active voice = 0).

These methods were considered but not implemented due to time constraints and the complexity of natural language processing required for robust implementation. However, they demonstrate the breadth of steganographic techniques available in text-based media.

Critical Analysis (Security and Threat Model)

❖ Assumptions

For this project, several assumptions were made to simplify the threat model:

- Network Integrity: The transfer of the carrier file across the network is assumed to be 100% accurate, with no loss or corruption of bits during

transmission. Packet loss, bit flips, or network errors could corrupt the hidden data.

- Key Distribution: A secure method of sharing the encryption key is assumed to exist, whether through a physical hand-off, a secure messaging app, or a pre-shared secret. The problem of key distribution is not solved by this project.

Mitigations:

- Integrity: To nullify the assumption of network efficiency, one could implement checksums or cryptographic hashes (e.g., SHA-256) and embed those alongside the secret. The recipient could then verify the integrity of the extracted message, satisfying the Integrity principle of the CIA triad.

- Key Distribution: To nullify the assumption of secure key distribution, one could use asymmetric cryptography (RSA) to encrypt the symmetric key (Fernet key) using the recipient's public key. The encrypted session key could be transmitted alongside the steg file, ensuring that only the intended recipient can decrypt the secret.

❖ Detection

Steganalysis is the art and science of detecting that hidden message, and if possible, extracting or destroying it.

To understand steganalysis, it helps to first remember what steganography is: hiding a secret message (like a text file, image, or malware) inside an innocent-looking carrier file (like a JPEG photo, MP3, or video) so that no one even knows a message exists.

Since steganography is a well-known method, there are a handful of techniques to break its purpose.

- Passive Steganalysis (Detection)

  Before breaking the code, an analyst must prove a secret exists. Several techniques are used, for example Visual/Perceptual Attacks (The "Eyeball" Test).

  For image steganography (LSB embedding), the human eye can sometimes spot anomalies, like when viewing only the Least Significant Bit (LSB) plane of an image, in a natural image, the LSBs should look like random noise. If the LSB plane shows a distinct, structured pattern (like a block of text or a clear outline), it is a dead giveaway.

  Another way is Contrast Enhancement. Steganography often dulls colors or creates unnatural flat areas. Raising the contrast or gamma of a suspicious image can reveal faint outlines of hidden data.

(a) Cover image (b) Stego-image after 50 % embedding (c) Stego-image after 100 % embedding

(An example of Visual/Perceptual Attack)

- Active Steganalysis (Extraction & Breaking)

  Once a hidden payload is detected, the analyst needs to extract it. However, the payload is almost always encrypted. Several methods are used:

The "Known Carrier" Attack (Dictionary Attack):

If the steganography software used is known (e.g., OpenStego, OutGuess, or DeepSound), the analyst knows exactly how the data was embedded. They simply run the same software on the suspicious file, trying out millions of common passwords (dictionary attack) until the software successfully decrypts and extracts the hidden file.

Brute-Force Key Search:

If the password isn't in a dictionary, analysts attempt to try every possible combination of characters. With modern GPUs, they can try billions of passwords per second, though this becomes impossible if the user uses a 128-character random key.

The "BMP to JPEG" Attack (Destructive):

This doesn't extract the message but destroys it so the recipient can't read it. Steganography relies on the exact data bits. If you take a suspected image and re-save it as a JPEG with maximum compression, or rotate it by 1 degree and save it, you scramble the pixel structure. This effectively corrupts the hidden payload, rendering it unrecoverable.

- Advanced Machine Learning (Universal Steganalysis)

  Older steganalysis targeted one specific embedding algorithm. Modern steganalysis uses Machine Learning (Support Vector Machines and Deep Neural Networks) to spot hidden data regardless of the algorithm used.

  Feature Extraction: The algorithm scans thousands of "subbands" (high-frequency and low-frequency areas) of an image using mathematical filters called Wavelet Transforms (specifically, the CF set or SPAM features).

The Anomaly: In a natural image, adjacent pixels have predictable mathematical relationships. Steganography breaks these relationships in a way that is imperceptible to humans. The ML model has been trained on millions of clean and stego-images and can spot this subtle "statistical drift" with over 95% accuracy, even when the message is less than 5% of the file size.

- Cutting-Edge Attacks (The Arms Race)

  The "Resampling" Attack (JPEG Ghosts):

  When you hide data in a JPEG, you have to save the image twice. Analysts use a technique called Error Level Analysis (ELA). By re-saving the image at varying quality levels and comparing the compression errors, they can spot areas of the image that were "touched" more recently than others. The location of the hidden data often reveals itself in these mismatched blocks.

  Cover-Source Mismatch (The "Printer" Fingerprint):

  Every digital camera, scanner, and printer leaves a unique noise pattern (Sensor Pattern Noise - SPN) on the image. If an analyst extracts the camera's unique fingerprint from a suspect image and finds the fingerprint is missing or distorted in certain color channels, they know those channels were overwritten with hidden data.

- The Ultimate Reality Check (The Encryption Problem)

  Here is the hard truth about modern steganalysis:

  Breaking the steganography is easy; breaking the encryption is not.

99% of steganography tools encrypt the payload with AES-256 before hiding it. Steganalysis can tell you exactly where the hidden data starts and ends in the file (e.g., "The secret starts at byte 14,502").

But without the password, you just have a chunk of high-entropy gibberish. You cannot read it. In the real world, forensic steganalysis rarely extracts the actual message via math. Instead, it extracts the hidden ciphertext, and then law enforcement relies on rubber-hose cryptanalysis (getting the password from the suspect) or legal warrants to seize the encryption keys from the suspect device.

❖ How Vulnerable is this project

The project's LSB steganography is vulnerable to all the passive attacks described above. In particular:

Visual attacks would easily detect the skewed even/odd distribution in images and audio.

Zero-Width Characters are trivially detected by scanning non-printable Unicode characters, and while communicating via most mediums, they are stripped off the text since they are not visible.

File size anomalies could raise suspicion if the secret is large.

Mitigation: The encryption layer (Fernet/AES-128) ensures that even if the hidden data is detected and extracted, it remains unreadable without the key. This is the defense's in-depth strategy; steganography provides stealth; cryptography provides security.

The "Hard Part" (Implementation Challenges)

❖ Capacity

When implementing steganography, we deal with two components: a mask text (the innocent carrier) and the secret message (the data to hide). A key constraint is that the carrier must be large enough to accommodate the secret. Since we use LSB embedding across all mediums, every character of the secret (8 bits) requires 8 units of the carrier medium to hide. However, what constitutes a "unit" differs by medium.

- Capacity in text

  Text steganography differs significantly from image or audio. In images and audio, we modify the LSB of color values or frequency samples, changes that are imperceptible to human senses. In text, however, modifying the LSB of a character's ASCII value (unique bit sequence) would change it into a completely different character (e.g., "A" would become "B"), altering the visible meaning of the carrier.

  To avoid this, we use Zero-Width Characters (ZWC). These invisible Unicode characters are inserted between visible characters without altering the visible text. Each ZWC stores one bit.

  The encoder loops through the mask text, inserting one ZWC after each visible character. If the mask text is longer than the secret, we stop when the secret is fully embedded. If the mask text is shorter, we append the remaining ZWCs at the end.

  Capacity Formula:

  Capacity (bits) = len(mask_text) characters.

  A 100-character mask text can hide 100 bits (~12.5 characters of encrypted secret).

Limitation:

While this method can theoretically hide messages of any length by appending to the end, an excessively large file with very little visible text raises suspicion; the file size would be larger than expected for the visible content. To avoid suspicious, ratio between secret message and carrier file should be as possible, around 1%, to even by passing steganalysis, as the charts would not be able to express that small of a change effectively.

- Capacity in image

The image is composed of pixels. Each pixel has three color channels: Red, Green, and Blue. Each channel is an 8-bit value (0-255), and we can modify the LSB of each channel to hide one bit. Therefore, one pixel can store 3 bits.

Capacity Formula:

Capacity (bits) = width × height × 3.

A 1920×1080 image has 1920 × 1080 × 3 = 6,220,800 bits (~777 KB).

Example: A sentence like "Kanye West is weird" (19 characters) requires 19 × 8 = 152 bits. This needs 152 / 3 = 51 pixels—a tiny fraction of a typical image.

- Capacity in audio

An audio sample is a single value representing the sound wave amplitude at a specific moment in time. Each sample is a 16-bit integer. We modify the LSB of each sample to store one bit. Therefore, one sample stores 1 bit.

Capacity Formula:

Capacity (bits) = sample_rate × duration × channels.

A 10-second WAV file at 44.1kHz with 2 channels has 44,100 × 10 × 2 = 882,000 bits (~110 KB).

Example: A 10-second audio clip can store approximately 110 KB of hidden data.

❖ Format Issues: Lossy vs Lossless

The most significant implementation challenge was handling lossy vs lossless formats.

Format	Type	LSB Integrity	Issue
PNG	Lossless	Preserved	Safe for LSB embedding.
JPEG	Lossy	Destroyed	Re-compression modifies pixel values, corrupting LSB data.
WAV	Lossless	Preserved	Safe for LSB embedding.
MP3	Lossy	Destroyed	Psychoacoustic compression modifies sample values, corrupting LSB data.

The Problem: JPEG and MP3 use lossy compression algorithms that sacrifice imperceptible details to reduce file size. Since LSB modifications are imperceptible, compression algorithms treat them as "noise" and overwrite them.

The Solution: All output files are forced to use lossless formats. PNG for images and WAV for audio. This ensures the embedded data survives intact.

Refection

The biggest implementation challenge was understanding the WAV file format. Unlike images, where Pillow handles the header automatically, audio required manually preserving the 44-byte header while modifying only the sample data. A single corrupted byte in the header would make the file unplayable.

Other challenges included:

- Bitwise operations: Understanding | 1 (set LSB to 1) and & 0xFE (clear LSB to 0) on signed 16-bit integers.

- Capacity planning: Ensuring the carrier was large enough before embedding.

- Platform compatibility: ZWC stripping on platforms like Gmail and Slack.

If I had more time, I would:

1. Add MP3 support using pydub or mp3stego-lib for real-world compatibility.
2. Implement synonym substitution to avoid ZWC stripping.
3. Add a honeypot feature for plausible deniability (a fake secret that decrypts with a second key).
4. Build a Streamlit UI for non-technical users.

What I learned:

Steganography is not just about hiding data, it's about understanding file formats, statistical detection, and the constant arms race between hiders and detectors. The tools work, but they are fragile and easily defeated by modern steganalysis.